Reification
In RDF 1.2

Boris Pelakh

May 2026

Why Reification? — Metadata on Facts

- **Provenance tracking** — record where a fact came from (source dataset, sensor, publication)
- **Confidence & uncertainty** — attach probability scores to uncertain statements
- **Temporal metadata** — record when a fact was valid or first asserted
- **Non-asserting belief statements** — describe a triple (e.g. “Alice believes X”) without adding it to the graph
- **Access control & governance** — tag triples with sensitivity, data classifications, or usage rights

RDF 1.2 Reification — The `reifies` Predicate

- A **Triple Term** names an RDF triple as a resource: `<< s p o >>`
- The `rdf:reifies` predicate links a reification node to the Triple Term it describes

N-Triples — asserting reification:

```
<http://ex.org/alice> <http://ex.org/knows> <http://ex.org/bob> .
_:r1 <...rdf-syntax-ns#reifies>
    <<( <http://ex.org/alice> <http://ex.org/knows> <http://ex.org/bob> )>> .
_:r1 <http://ex.org/confidence> "0.95"^^xsd:decimal .
_:r1 <http://ex.org/source> <http://dbpedia.org/> .
```

Turtle — explicit reifies predicate:

```
:r1 rdf:reifies <<( :alice :knows :bob )>> ;
    :confidence 0.95 ;  :source :DBpedia .
```

Turtle — Annotation & Subject-Graph Syntaxes

Annotation syntax — base triple IS asserted:

```
<< :alice :knows :bob >> :confidence 0.95 ; :source :DBpedia .
```

- Creates a blank reification node; the base triple is asserted

Annotation syntax — explicit :

```
<< :alice :knows :bob  ~ :stmt1 >> :confidence 0.95 ; :source :DBpedia .
```

- The IRI of the statement (`rdf:Proposition`) is explicitly specified

Subject-graph syntax — also asserts the base triple:

```
:alice :knows :bob {| :confidence 0.95 |} .
```

- Shorthand for annotation syntax; base triple is also asserted

Non-asserting — explicit rdf:reifies (base triple NOT asserted):

```
:r1 rdf:reifies <<( :alice :knows :bob )>> ;
    :confidence 0.95 .
```

- Only the Triple Term is referenced; `:alice :knows :bob` is NOT added to the graph

Asserting vs. Non-Asserting Reification

Syntaxes that assert the base triple:

- Use when the fact IS true and you also want to record metadata about it
- Example: confirmed sensor reading with provenance

Non-asserting — describes without adding:

- Explicit `rdf:reifies` on a named node does NOT assert the base triple
- Use for beliefs, hypotheticals, disputed claims, or annotations by a third party
- Example: "Alice believes :x :p :y" — without claiming :x :p :y is actually true

Key rule:

- A Triple Term `<<( s p o )>>` is only an identifier — does not add `s p o` to the dataset unless explicitly asserted

N-Triples Format for Reification

- N-Triples uses <<(s p o)>> for Triple Terms
- Each statement is one line, terminated with . — no syntactic sugar for annotation

Asserting reification — base triple stated first:

```
<http://ex.org/alice> <http://ex.org/knows> <http://ex.org/bob> .
_:r <http://www.w3.org/1999/02/22-rdf-syntax-ns#reifies>
   <<( <http://ex.org/alice> <http://ex.org/knows> <http://ex.org/bob> )>> .
_:r <http://ex.org/source> <http://dbpedia.org/> .
```

Non-asserting — base triple omitted:

```
_:b <http://www.w3.org/1999/02/22-rdf-syntax-ns#reifies> <<(
<http://ex.org/alice> <http://ex.org/knows> <http://ex.org/bob> )>> .
_:b <http://ex.org/assertedBy> <http://ex.org/Carol> .
```

RDF 1.2 Reification vs. LPG Edge Properties

LPG (Labeled Property Graph) edge properties:

- Each edge has a transient, system-assigned ID (not a first-class entity)
- Edge properties can only be scalars: strings, numbers, booleans
- An edge property cannot link to another node — no traversal from edge metadata
- The edge cannot be referenced as a subject or object in another relationship
- **Cannot assert properties about a non-asserted edge**

RDF 1.2 Triple Terms are first-class resources:

- A Triple Term `<< s p o >>` is a persistent, referenceable resource (like a URI or blank node)
- Reification metadata can be any RDF triples — including links to other named entities
- Multiple independent reifications can describe the same underlying triple
- Triple Terms can themselves be subjects of further triples (nested metadata)

RDF 1.2 vs. LPG — Side-by-Side Comparison

LPG — edge property (Neo4j Cypher):

```
(alice)-[:KNOWS {
  confidence: 0.95,
  source: 'DBpedia'   // scalar string
}]->(bob)
```

- 'source' is a string literal — cannot traverse to a `:DBpedia` node or query its properties

RDF 1.2 — reification with linked metadata (Turtle):

```
:alice :knows :bob
  {| :confidence 0.95 ;
     :source :DBpedia ;       // URI — traversable
     :citedIn :Paper42 |} .  // link to another entity
```

- 'source' is a URI: traverse to `:DBpedia` and query its own properties
- 'citedIn' links to a publication entity, enabling rich provenance subgraphs

SPARQL Queries for Reification — Basic Retrieval

Retrieve all metadata for a specific triple:

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX ex:  <http://example.org/>

SELECT ?reif ?prop ?value WHERE {
  ?reif rdf:reifies << ex:alice ex:knows ex:bob >> ;
        ?prop ?value .
}
```

- Returns every property of every reification node describing this specific triple
- Works for both asserting and non-asserting reifications
- Inline Triple Term `<< ex:alice ex:knows ex:bob >>` is a constant in the query; can also bind with variables

SPARQL — Filtering Triples by Metadata

Find all high-confidence statements from a trusted source:

```
SELECT ?s ?p ?o WHERE {
  ?r rdf:reifies <<(?s ?p ?o)>> ;
     ex:confidence ?conf ;
     ex:source ex:TrustedDB .
  FILTER (?conf >= 0.9)
}
```

- Variable Triple Terms `<<(?s ?p ?o)>>` bind subject, predicate and object of the described triple

Find triples Alice believes that are NOT yet asserted:

```
SELECT ?s ?p ?o WHERE {
  ?r rdf:reifies <<(?s ?p ?o)>> ;
     ex:believedBy ex:alice .
  FILTER NOT EXISTS { ?s ?p ?o }
}
```

- `FILTER NOT EXISTS` checks the base triple is absent from the graph

SPARQL — Traversing Provenance Chains

Retrieve triples with full linked provenance:

```
SELECT ?s ?p ?o ?srcName ?paper WHERE {
  << ?s ?p ?o >>
     ex:source ?src ;
     ex:citedIn ?paper .
  ?src rdfs:label ?srcName .
}
```

- Because `ex:source` is a URI, you can join to `?src` and retrieve any of its properties
- Impossible in LPG where edge properties are scalars and cannot be traversed

Count supporting sources per triple:

```
SELECT ?s ?p ?o (COUNT(?r) AS ?n) WHERE {
  ?s ?p ?o {| ex:source ?src |} .
} GROUP BY ?s ?p ?o ORDER BY DESC(?n)
```


semantic arts

Summary

- RDF 1.2 adds native reification via `rdf:reifies` and Triple Terms `<< s p o >>`
- Three Turtle syntaxes: explicit `rdf:reifies`, annotation `<< … >>`, subject-graph `{| ... |}`
- Annotation and subject-graph syntaxes assert the base triple; explicit `rdf:reifies` is used when the base triple is not asserted.
- RDF 1.2 vs. LPG: Triple Terms are first-class resources; metadata can be URI links, not just scalars
- Multiple independent reifications per triple; nested metadata supported
- SPARQL `<<(?s ?p ?o)>>` variable Triple Terms enable powerful metadata and provenance queries